

Scaling Laws: 从训练到推理

韩成瑞

东南大学数学学院

2026 年 6 月 3 日



目录

- 1 引言
- 2 论文一：Kaplan et al. (2020) 幂律缩放
- 3 论文二：Chinchilla (2022) 计算最优训练
- 4 论文三：推理 Scaling Laws (2025)
- 5 综合比较与总结

目录

- 1 引言
- 2 论文一：Kaplan et al. (2020) 幂律缩放
- 3 论文二：Chinchilla (2022) 计算最优训练
- 4 论文三：推理 Scaling Laws (2025)
- 5 综合比较与总结

一个贯穿始终的核心问题

中心问题

在固定算力预算下，如何分配计算资源才能获得最优模型质量？

三篇论文从不同角度回答了这个问题：

- ① **Kaplan et al. (2020)** ——发现规律：损失与规模之间存在稳定的幂律关系。
- ② **Hoffmann et al. (2022, Chinchilla)** ——优化训练：在固定算力下找到最优的模型-数据配比。
- ③ **Wu et al. (2025, 推理缩放)** ——优化推理：将测试时算力纳入优化框架，寻找最优推理策略。

主线逻辑

训练缩放告诉我们如何训练一个好模型；推理缩放告诉我们如何高效使用一个已有模型。

符号	含义
N	非嵌入参数数量
D	训练 token 数量
C_{train}	训练计算量 (FLOPs)
C_{infer}	单次查询的推理计算量
L	验证集交叉熵损失

对稠密 decoder-only Transformer，最常用的计算量估计为：

$$C_{\text{train}} \approx 6ND \quad (1)$$

其中系数 6 来自每个 token 的前向传播 ($\approx 2N$) 加上反向传播 ($\approx 4N$)。

三篇论文的核心区别

Kaplan (2020)

- 问题：损失如何随 N 、 D 、 C 变化？
- 方法：拟合幂律曲线。
- 结论：规模越大，效果越好，且可预测。

Chinchilla (2022)

- 问题：固定 C 时，最优的 N 和 D 如何选取？
- 方法：大规模网格实验 + 拉格朗日优化。
- 结论：数据应与模型规模等比增长。

推理缩放 (2025)

- 问题：固定模型后，如何分配测试时算力？
- 方法：实验 + 理论收敛分析。
- 结论：推理策略比单纯扩大模型更高效。

共同数学本质

三篇论文的核心结构都是**资源约束下的边际收益递减**：单位 FLOP 带来的性能提升会逐渐减小。

目录

- 1 引言
- 2 论文一：Kaplan et al. (2020) 幂律缩放
- 3 论文二：Chinchilla (2022) 计算最优训练
- 4 论文三：推理 Scaling Laws (2025)
- 5 综合比较与总结

核心经验结论：损失服从幂律

Scaling Laws for Neural Language Models 在数个数量级的规模上系统测量了损失变化。

三个独立的幂律关系——性能仅由单一变量决定时：

- 模型缩放： $L(N) \approx L_\infty + AN^{-\alpha_N}$, $\alpha_N \approx 0.076$
- 数据缩放： $L(D) \approx L_\infty + BD^{-\alpha_D}$, $\alpha_D \approx 0.095$
- 算力缩放： $L(C) \approx L_\infty + EC^{-\alpha_C}$, $\alpha_C \approx 0.05$

为什么叫" 幂律"?

在对数坐标 (log-log) 下, $\log L \propto -\alpha \log N$, 表现为直线——性能提升平滑、持续且跨越数个量级可预测。

L_∞ 是不可约的误差下界, 代表人类语言本身的内在不确定性。

Kaplan 的算力分配建议

已知 $C_{\text{train}} \approx 6ND$ ，在固定算力预算 C_0 下，增大 N 必然压缩 D ，反之亦然——本质上是一个分配问题。

Kaplan 等根据其幂律拟合，给出的最优分配（渐近意义下）：

$$N_{\star} \propto C_0^{0.73}, \quad D_{\star} \propto C_0^{0.27}. \quad (2)$$

- 偏向于扩大模型：当算力翻倍时，最优参数量增长约 65%，而数据仅增长约 21%。
- 含义：可以适当早停（不必等模型完全收敛），将节省的 FLOPs 用于增大 N 。

遗留问题（为论文二埋下伏笔）

Kaplan 的实验规模相对有限，其拟合指数是否在更大算力下仍然成立？后续 Chinchilla 给出了修正答案。

Kaplan 确立的核心事实

损失服从幂律，且由 $C_{\text{train}} = 6ND$ 约束，资源分配决定性能上限。

但 Kaplan 的分析有一个关键缺口：

- 实验覆盖的参数量/数据量范围尚小，指数拟合存在外推误差。
- 真正"最优"的 (N_*, D_*) 需要同时改变 N 和 D ，而不是分别测量。

论文二的贡献：在更宽的参数网格上做联合实验，给出严格的二维最优优化答案。

目录

- 1 引言
- 2 论文一：Kaplan et al. (2020) 幂律缩放
- 3 论文二：Chinchilla (2022) 计算最优训练
- 4 论文三：推理 Scaling Laws (2025)
- 5 综合比较与总结

问题设定：二维联合优化

目标：在固定训练算力 C_0 下，同时选择 N 和 D ，使验证损失最小。

$$\min_{N,D} L(N,D) \quad \text{s.t.} \quad C_{\text{train}} = 6ND \leq C_0 \quad (3)$$

Hoffmann 等提出了一个可分离的损失模型：

$$L(N,D) = \underbrace{E}_{\text{不可约下界}} + \underbrace{\frac{A}{N^\alpha}}_{\text{参数受限项}} + \underbrace{\frac{B}{D^\beta}}_{\text{数据受限项}} \quad (4)$$

直觉解释：

- E ：即使模型无限大、数据无限多也无法消除的误差（语言本身的随机性）。
- 第二项：模型太小，容量瓶颈带来的损失。
- 第三项：训练数据太少，泛化不足带来的损失。

拉格朗日推导：最优平衡条件

在等式约束 $6ND = C_0$ 下，构造拉格朗日函数：

$$\mathcal{J}(N, D, \lambda) = E + AN^{-\alpha} + BD^{-\beta} + \lambda(6ND - C_0).$$

令偏导数为零，得到一阶最优性条件：

$$\frac{\partial \mathcal{J}}{\partial N} = 0 \Rightarrow \alpha AN^{-\alpha-1} = 6\lambda D, \quad (5)$$

$$\frac{\partial \mathcal{J}}{\partial D} = 0 \Rightarrow \beta BD^{-\beta-1} = 6\lambda N. \quad (6)$$

消去 λ ，得到核心最优平衡条件：

$$\boxed{\alpha AN^{-\alpha} = \beta BD^{-\beta}} \quad (7)$$

含义：最优点处，"再多一个参数带来的损失下降" = "再多一个 token 带来的损失下降"。

最优缩放率的闭式解

联立平衡条件与约束 $ND = C_0/6$, 可以解出:

$$N_{\star} \propto C_0^{\frac{\beta}{\alpha+\beta}}, \quad D_{\star} \propto C_0^{\frac{\alpha}{\alpha+\beta}}. \quad (8)$$

论文通过大规模实验拟合得到 $\alpha \approx \beta$, 因此两个指数都接近 $1/2$:

$$N_{\star} \sim C_0^{0.5}, \quad D_{\star} \sim C_0^{0.5}.$$

与 Kaplan 的关键差异

Kaplan 给出 $N_{\star} \propto C_0^{0.73}$, Chinchilla 给出 $N_{\star} \propto C_0^{0.5}$ 。

含义: 在算力翻倍时, Chinchilla 建议更激进地增加数据量, 而不是一味扩大模型。

实用经验法则:

$$D_{\star} \approx 20N_{\star} \text{ tokens}$$

(9)

为什么 GPT-3 / Gopher 属于欠训练?

在 Chinchilla 框架下审视当时 (2020–2022 年) 主流模型:

模型	参数量 N	训练 token D	D/N
GPT-3 (OpenAI, 2020)	175B	300B	≈ 1.7
Gopher (DeepMind, 2021)	280B	300B	≈ 1.1
Chinchilla 最优	N	20N	20

当时模型的 D/N 远小于 20, 即数据严重不足——模型容量买了很多, 但没有足够数据来填满它。

后果:

- 用同等 FLOPs 可训出更小、性能相当的模型, 推理成本更低。
- 数据效率差, 每个 token 没有得到充分利用。

Chinchilla 的实证验证

DeepMind 用 70B 参数 + 1.4T token 训练了 Chinchilla 模型，与 Gopher (280B, 300B tokens) 对比：

模型	参数量	训练 token	多数评测结果
Gopher	280B	300B	基线
Chinchilla	70B	1.4T	持平或更优

尽管参数量减少了 $4\times$ ，Chinchilla 在大多数评测任务上表现更好。

工程启示

在相同的训练 FLOPs 下，"训练一个更小但数据更充分的模型" 往往优于 "训练一个更大但数据不足的模型"，且推理成本更低。

Chinchilla 的落地流程

给定训练预算 C_0 ，完整的计算最优训练流程：

- ① 拟合缩放参数：在小规模网格实验（不同 N, D 组合）上拟合 E, A, B, α, β ：

$$L(N, D) = E + AN^{-\alpha} + BD^{-\beta}.$$

- ② 求最优配比：由 KKT 条件联立 $6ND = C_0$ ，求得 (N_*, D_*) 。
- ③ 验证并训练：在 (N_*, D_*) 附近训练，并在验证集上确认预测与实测吻合。
- ④ 迭代更新：若架构或数据分布变化，重新拟合并迭代。

论文二的核心价值

将 Scaling Law 从"描述性规律" 升级为"可操作的优化工具"——给定预算，直接输出最优配比。

Chinchilla 的适用范围与局限

Chinchilla 框架被广泛采用，但有几点重要前提需要注意：

- 架构依赖：拟合的指数 α, β 对不同架构（如 MoE、多模态）可能不同，需要重新拟合。
- 数据质量未建模：损失函数中的 D 只计数 token，忽略了数据质量差异（如网页文本 vs 书籍）。
- 优化的是平均损失：不保证在所有下游任务（如推理、代码）上都最优。
- 内存和延迟被忽略：实际部署时，推理成本更多受模型大小而非训练数据量影响。

尽管如此，其核心原则保持稳健：

让" 再增加一个参数" 与" 再增加一个 token" 带来的边际收益相等。

从训练优化到推理优化——过渡

论文二解决了训练阶段的资源分配问题。但模型训练完成后，还面临第二个资源分配阶段——推理。

考虑全生命周期计算：

$$\text{总计算} = C_{\text{train}} + Q \cdot C_{\text{infer}}, \quad (10)$$

其中 Q 是服务的查询次数。当 Q 很大时，推理成本不可忽视。

新问题： 对于一个已训练好的模型，每次推理时如何分配计算资源（生成更多候选？用更好的搜索策略？）才能在同等推理预算下获得最优性能？

目录

- 1 引言
- 2 论文一：Kaplan et al. (2020) 幂律缩放
- 3 论文二：Chinchilla (2022) 计算最优训练
- 4 论文三：推理 Scaling Laws (2025)
- 5 综合比较与总结

Inference Scaling Laws: An Empirical Analysis of Compute-Optimal Inference for LLM Problem-Solving

ICLR 2025, Wu et al. (清华 + CMU)

研究设置:

- 任务: 数学推理 (GSM8K、MATH500、MATH), 有确定性正确答案, 方便量化准确率。
- 模型: Pythia、Llemma、Mistral 系列 (覆盖不同参数量)。
- 推理策略: 从简单到复杂——greedy、best-of-n、majority voting、weighted voting、MCTS、**REBASE**。

核心实验发现

在低到中等推理预算下, 增加推理算力 (更多采样、更好搜索) 比增大模型参数量更具算力效率。

形式化：计算最优推理问题

将推理性能误差定义为：

$$E(N, T; S), \quad (11)$$

其中 N 是模型规模， T 是每次推理生成的 token 总数， S 是推理策略（如 majority voting 或 REBASE）。

推理计算量由三者共同决定：

$$C = \text{FLOPs}(N, T; S). \quad (12)$$

计算最优推理问题：

$$(N_{\text{opt}}, T_{\text{opt}}; S) = \arg \min_{\substack{(N, T, S) \\ \text{FLOPs}(N, T; S) = C}} E(N, T; S). \quad (13)$$

关键点：最优模型大小 N_{opt} 会随推理预算 C 变化——并非越大越好。

推理策略：从简单采样到树搜索

策略	方法	特点
Greedy	单次贪心生成	基准，FLOPs 最少
Best-of-n	采样 n 次，选最高分	需要奖励模型
Majority Voting	采样 n 次，多数答案	不需要额外模型
Weighted Voting	答案按奖励加权投票	需要奖励模型
MCTS	树搜索 + 价值估计	搜索覆盖广但 token 浪费多
REBASE	过程奖励引导的平衡搜索	本文提出， Pareto 最优

核心问题：同等推理 FLOPs 下，哪种策略（以及多大模型）能给出最低误差？

理论基础：多数投票的收敛性（定理 1）

设测试集 $\mathcal{D} = \{(x_i, y_i)\}_{i=1}^m$ ，策略 π 对每个问题 x_i 的答案分布为：

$$G_i(y) = \sum_{r \in V^*} \pi(r \text{ e } y \mid x_i), \quad (14)$$

即通过所有推理链 r 到达答案 y 的总概率（e 为分隔符 token）。

多数投票准确率的渐近行为：

$$\lim_{n \rightarrow \infty} \text{acc}_n^{\text{MV}}(\mathcal{D}; \pi) = \frac{1}{m} \sum_{i=1}^m \mathbf{1} \left[y_i = \arg \max_{|y| \leq L} G_i(y) \right] \quad (\text{a.s.}) \quad (15)$$

且期望准确率以指数速度收敛到极限：

$$\mathbb{E}[\text{acc}_n^{\text{MV}}] = \text{极限} - O(c^{-n}), \quad c > 1.$$

含义：多数投票的上界是固定的（由模型 π 决定），采样再多也无法突破此上界。

理论基础：加权投票的收敛性（定理 2）

引入过程/结果奖励模型 ρ ，加权后的答案分布为：

$$G_i^\rho(y) = \sum_{r \in V^*} \pi(r \text{ e } y \mid x_i) \cdot \rho(x_i, r, e, y). \quad (16)$$

类似地，加权投票也以指数速度收敛到一个极限：

$$\lim_{n \rightarrow \infty} \text{acc}_n^{\text{WV}}(\mathcal{D}; \pi, \rho) = \frac{1}{m} \sum_{i=1}^m \mathbf{1} \left[y_i = \arg \max_{|y| \leq L} G_i^\rho(y) \right] \quad (\text{a.s.}) \quad (17)$$

两个定理的关键推论

- 当奖励模型 ρ 有效时，加权投票的渐近上界严格高于普通多数投票。
- 但两者都存在饱和上界——仅靠增加采样数，边际收益以 $O(c^{-n})$ 速度趋零。
- 这从理论上说明：需要更聪明的搜索策略（而非单纯多采样）来突破上界。

REBASE: 突破饱和上界的搜索策略

为什么 **MCTS** 不够好? MCTS 需要大量 rollout 来估计节点价值, 许多计算花在了不会成为最终答案的路径上, token 利用率低。

REBASE (REward BALanced SEarch) 的设计思路:

- 用过程奖励模型 (**PRM**) 对搜索树中间节点打分, 而非等到生成完整答案。
- 按 softmax 归一化奖励, 自适应地分配每层的扩展宽度:

$$W_j = \text{Round} \left(B_i \cdot \frac{\exp(R(n_j)/T_b)}{\sum_k \exp(R(n_k)/T_b)} \right). \quad (18)$$

其中 B_i 是当前层的总扩展预算, $R(n_j)$ 是节点 n_j 的奖励分数, T_b 是温度超参数。

效果: 优质路径获得更多计算资源, 劣质路径被早期剪枝, 在同等 **FLOPs** 下生成更多高质量完整候选答案。

经验结果：小模型 + 好策略可胜大模型

在 MATH / GSM8K 上，论文报告了 Pareto 前沿分析：

代表性结论：

$$\underbrace{\text{Llemma-7B} + \text{REBASE}}_{\text{小模型} + \text{智能搜索}} \succ \underbrace{\text{Llemma-34B} + \text{standard voting}}_{\text{大模型} + \text{简单策略}}$$

在相同推理 FLOPs 预算下成立。

其他重要发现：

- 最优模型大小随预算变化：推理 FLOPs 少时，应选小模型 + 多采样；预算充足时，大模型更合适。
- **REBASE** 位于成本-性能前沿：在各预算区间均优于或持平其他策略。
- **greedy** 的性价比陷阱：单次生成 FLOPs 少，但准确率低，总体效率不高。

论文三的逻辑链

论文三构建了一条从形式化到算法的完整逻辑链：

- ① 形式化：将计算最优推理定义为 (N, T, S) 上的约束最小化问题。
- ② 理论分析：证明采样/投票方法以指数速度收敛到固定上界（定理 1、2）。
- ③ 结构性局限：仅靠多采样存在不可突破的饱和上界，需要更好的搜索策略。
- ④ 算法设计：提出 REBASE，在同等 FLOPs 下提升成本-性能效率。

核心洞见

推理阶段的算力分配是一个算法设计问题，而不仅仅是"多采样几次"。选择正确的推理策略，其收益可以超过单纯增大模型规模。

目录

- 1 引言
- 2 论文一：Kaplan et al. (2020) 幂律缩放
- 3 论文二：Chinchilla (2022) 计算最优训练
- 4 论文三：推理 Scaling Laws (2025)
- 5 综合比较与总结

训练缩放 vs 推理缩放：统一视角

维度	训练缩放（论文一/二）	推理缩放（论文三）
优化变量	(N, D)	$(N, T; S)$
预算约束	$C_{\text{train}} = 6ND$	$\text{FLOPs}(N, T; S) \leq B$
损失模型	$L = E + AN^{-\alpha} + BD^{-\beta}$	策略相关的收敛上界
最优原则	模型项与数据项边际收益相等	在饱和前按边际收益分配预算
典型失误	欠训练（ D 太少）	在简单问题上过度消耗推理算力
关键工具	拉格朗日优化 + 网格实验	过程奖励模型 + 树搜索

统一原则

无论是训练还是推理，最优分配的数学本质相同：将资源分配到单位 **FLOP** 边际收益最高的地方，直到各方向边际收益相等。

- ① 预训练前：用小规模 pilot 实验拟合缩放指数，预测大规模训练的最优 (N_* , D_*)。
- ② 预训练中：按 Chinchilla 风格分配资源，避免过度增大参数但忽视数据量。
- ③ 部署时：根据任务类型和查询量选择推理策略；对难题使用 REBASE 等搜索策略，对简单查询使用 greedy 以节省成本。
- ④ 持续监控：记录推理成本-性能曲线，发现饱和点后及时切换到更好的策略，而非盲目增加采样次数。

训练 vs 推理的分工

训练缩放解决容量规划（买多大的模型）；推理缩放解决单次查询控制（每次推理花多少算力）。两者都遵循相同的边际收益原则。

核心结论回顾

- ① **Kaplan (2020)**: 缩放规律真实存在——损失与模型、数据、算力均服从幂律，性能提升可预测。
- ② **Chinchilla (2022)**: 计算最优训练显著偏向"多数据"——最优 N 和 D 应同步增长 ($D \approx 20N$)，此前模型普遍欠训练。
- ③ **推理缩放 (2025)**: 测试时算力也遵循自身规律——采样方法存在理论饱和上界，REBASE 等搜索策略能在同等 FLOPs 下突破此上界。

三篇论文的统一主题

在任何约束下，都应把算力分配到**边际收益最高的地方**。这一原则在训练和推理阶段通用，且有严格的数学保障。

- 架构泛化性：缩放指数在 MoE、多模态、状态空间模型下有多稳定？
- 数据质量建模：能否在 Scaling Law 中显式引入数据质量指标？
- 联合优化：能否同时优化训练语料质量与推理阶段搜索策略？
- 任务相关性：数学推理的推理缩放结论能否推广到代码、多模态、规划等任务？

终极愿景：从预训练到部署，实现跨阶段的全生命周期算力最优化。



Kaplan J, McCandlish S, Henighan T, et al.
Scaling Laws for Neural Language Models.
arXiv preprint arXiv:2001.08361, 2020.



Hoffmann J, Borgeaud S, Mensch A, et al.
Training Compute-Optimal Large Language Models.
arXiv preprint arXiv:2203.15556, 2022.



Wu Y, Sun Z, Li S, Welleck S, Yang Y.
Inference Scaling Laws: An Empirical Analysis of Compute-Optimal Inference
for LLM Problem-Solving.
ICLR, 2025.

Thanks!